

A Discipline of Partial Audit Classification

On the Correct Treatment of Undefined Audit Assertions

Duston Moore

the date of receipt and acceptance should be inserted later

Abstract Audit systems sometimes use unverifiable assertions. This is a classification error, and it has a precise form, close enough to a type error to justify the analogy throughout this paper: an assertion classified `Undefined` is used where an assertion classified `Verified` is required. This paper develops a discipline that makes the error structurally inadmissible and implements it as an executable pipeline: boundary classification computes `Verified` or `Undefined` from a declared, versioned boundary specification; independent admission is certificate-based, checked against a declared process registry rather than asserted by a bare role label; every material dependency that is not both classified `Verified` and validly admitted automatically writes a durable entry to an append-only residual register; and an extracted opinion-decision procedure is proved sound and complete: it returns an unqualified decision if and only if every material dependency is verified and validly admitted. The central invariant these mechanisms jointly enforce is stated once and used throughout: no audit conclusion may depend on a material assertion whose verification state is undefined. `Undefined` is presented as a success state: it is the correct output of a classifier that has refused to speak beyond its boundary. The Wirecard collapse is treated not as motivation but as a classification trace exhibiting a violation of the invariant, and continuous auditing exception queues and SQL's three-valued `Unknown` supply the same violation at ordinary scale, each exercised as a fixture run through the pipeline rather than described only in prose. The development is machine-checked in Coq/Rocq – a classification kernel and extracted reference implementation, not merely a sketch – and every stage is proved sound individually before the end-to-end soundness theorem composes them. The method does not prevent fraud, does not establish that supplied evidence is genuine or that supplied materiality judgements are correct, and does not prove that distinct registered identities are free of collusion or organisational dependence; evidence authenticity and materiality selection remain the caller's responsibility throughout. It prevents an audit classification system from treating an undefined material assertion as verified, which is a narrower and therefore provable claim.

D. Moore
Independent Researcher, Edmonton, Alberta, Canada
E-mail: dhwcmoore@gmail.com

Keywords audit classification · verification failure · partial functions ·
admissibility · three-valued logic · formal methods · accounting information systems

1 The Problem

An audit is, among other things, a classification system. It receives assertions together with evidence and it produces classifications on which conclusions depend. The problem addressed in this paper is that audit systems sometimes use unverifiable assertions, and that they do so silently. An assertion that could not be verified within the evidence, authority, and observation boundaries available to the system is nonetheless treated, downstream, as though it had been verified.

This is not a moral failure and it is not, in the first instance, a regulatory failure. It is close enough to a type error to be usefully treated as one: an assertion classified `Undefined` is being used where an assertion classified `Verified` is required. The analogy is deliberate and is used throughout this paper as an engineering description of the failure, not as a literal claim about the type system of any particular implementation language; where the accompanying Coq/Rocq development makes a genuinely type-theoretic version of the claim (Section 8), that is stated separately and more narrowly. The remainder of this paper is the discipline that makes the error structurally inadmissible.

Three terms are needed and are used consistently throughout. *Verification failure* is the condition in which a system cannot determine whether a claim is verified or refuted within its available boundaries. An *implicit default* is the local rule, convention, or implementation path that fills the resulting gap, typically with acceptance. *Silent conversion* is the system-level result: the unresolved claim is treated as accepted, and no explicit failure state records that this happened.

Audit standards already distinguish sufficient evidence from its absence, and provide qualified opinions and disclaimers precisely for cases of evidential insufficiency (IAASB, ISA 500 2016; IAASB 2016). The failure this paper targets is not that auditing lacks any concept of insufficient evidence; Section 9 argues at length that it does not. The failure is narrower and more specific: some audit workflows operationally behave as though an unresolved input had been accepted, even where the standard itself provides an explicit non-affirmative channel, because nothing in the classification *logic* of the surrounding system forces that channel to be taken rather than silently bypassed. The substantive partiality is in the verification procedure, not in the classifier built around it: a procedure such as direct bank confirmation is genuinely partial over its intended domain, defined only where its preconditions hold. The discipline of this paper does not leave that partiality unresolved at the point of audit consequence. It totalises it honestly, by returning an explicit `Undefined` result outside the verified domain rather than either an undefined program state or a convenient default:

<code>cannot verify -> accept</code>	<code>(undisciplined)</code>
<code>cannot verify -> Undefined</code>	<code>(disciplined)</code>

The whole paper is contained in the difference between those two lines. `Undefined` is not a defect of the disciplined classifier. It is the sign that the classifier has refused to lie.

One rule governs everything that follows.

Rule 1.1 *Do not say that the system provides assurance. Say exactly what proposition it verifies.*

The central invariant can now be stated. Everything else in the paper is support for it.

Rule 1.2 (Central Invariant) *No audit conclusion may depend on a material assertion whose verification state is undefined.*

In symbolic form:

```
Admissible(Opinion) =>
  for every material assertion a used by Opinion:
    state(a) = Verified
```

Equivalently, and more sharply:

```
state(a) = Undefined => a is inadmissible for an unqualified opinion
```

Contributions

This paper’s contribution is seven pieces built and machine-checked as one development, not seven separate claims:

1. a declared, typed audit-classification state model (Section 2);
2. a versioned boundary evaluation mechanism that computes `Verified` or `Undefined` from a declared specification and an evidence context, with a proved witness for every `Verified` outcome (Section 4, Section 8);
3. certificate-based admission separation: an admission certificate is checked against a declared process registry, not asserted by a bare role label (Section 3, Section 8);
4. automatic residual emission and append-only preservation for every material dependency that is not both verified and validly admitted (Section 5);
5. an executable opinion-decision procedure, proved sound and complete over the properties above (Section 8);
6. Rocq soundness theorems for every stage individually and for their composition into one end-to-end pipeline (Section 8);
7. an extracted OCaml implementation and reproducible fixtures exercising all of the above on three worked cases (Section 7, Section 8).

The paper proceeds by construction. Section 2 defines the state space. Section 3 defines admissibility. Section 4 defines boundaries and gives every verification procedure a precondition and a postcondition. Section 5 defines residuals and the register that preserves them. Section 6 gives the composition rule for opinions. Section 7 exhibits three violations of the invariant: Wirecard as a classification trace, continuous auditing exception queues as an algorithm, and SQL `Unknown` as a semantic error at the application layer. Section 8 gives the formal development. Section 9 states what existing literatures supply and what they do not. Section 10 draws consequences for accounting information systems, for standards, and for AI-assisted audit tools.

2 States

Loose words such as assurance, reasonable, sufficient, and appropriate are admitted into this paper only after they have been turned into typed states. The state space is declared once and used everywhere.

```

AuditAssertion ::=
  claim about existence
| claim about valuation
| claim about completeness
| claim about rights and obligations
| claim about control

EvidenceState ::=
  Verified
| Refuted
| Undefined
| PresumptivelyVerified
| PresumptivelyRefuted
| EscalationRequired

AuditUse ::=
  Admissible
| Inadmissible

```

The BNF above fixes the vocabulary informally; the Coq/Rocq development declares the same six constructors as an inductive type, so that the formal development of Section 8 is stated over exactly this vocabulary rather than a re-encoding of it:

```

Inductive EvidenceState : Type :=
| Verified
| Refuted
| Undefined
| PresumptivelyVerified
| PresumptivelyRefuted
| EscalationRequired.

Inductive AuditUse : Type :=
| Admissible
| Inadmissible.

```

The states have the following meanings and no others. **Verified**: the assertion falls within a declared boundary and the evidence grade that boundary requires is present. **Refuted**: a resolved negative classification – admissible evidence establishes that the assertion is false, not merely that it leans false. **Undefined**: the assertion falls outside all declared boundaries, or the required evidence grade is absent, and no admissible procedure currently resolves it. **PresumptivelyVerified** and **PresumptivelyRefuted**: available evidence leans one way without meeting the declared grade for the assertion’s materiality; these are unresolved, not resolved-in-either-direction. **EscalationRequired**: the assertion exposes a defect in the boundary specification itself, such as a conflict, an overlap, or an assertion class for which no boundary was declared.

Production and mechanisation are not the same claim, and this paper is explicit about the boundary between them, exactly matching the accompanying develop-

ment. Only two of the six states are produced by a declared, generic, machine-checked procedure: the boundary evaluation function of Section 4 (`classify` in the Coq/Rocq development) computes `Verified` when a declared procedure’s preconditions are satisfied by the evidence context, and `Undefined` otherwise, and no third branch exists for it to take. `Refuted`, the two presumptive states, and `EscalationRequired` are declared vocabulary with the meanings above; they are legitimate inputs to every downstream mechanism in this paper – composition (Section 6), residual emission (Section 5), and the opinion decision (Section 8) all accept a dependency already classified into any of the six states, and the theorems of Section 8 are stated generally enough to cover that case – but no generic, proved-correct procedure for *assigning* one of these four states from a boundary specification and an evidence context is formalised in the accompanying development. There is consequently no state precedence to state here: where an earlier informal presentation of this discipline suggested one, the machine-checked version does not implement it, and this paper states the narrower, implemented claim rather than the broader one.

`EscalationRequired` is declared vocabulary describing a defect in a boundary specification, not a workflow status a queue-management system would assign; nothing in this paper’s formal development computes it. The distinction between a classification state and a review status matters most for the continuous-auditing case (Section 7.2): a transaction’s *classification* is exactly one of the six states above, computed or declared; its *review status* (reviewed, queued, deprioritised) is workflow metadata the boundary machinery does not itself track. Section 7.2 accordingly does not introduce a separate `Unreviewed` classification state; a deprioritised transaction is `Undefined`, in the same sense and by the same mechanism as any other assertion outside its verified domain.

The only state transition this paper’s formal development models is the residual register’s append-only `step` relation (Section 5): new entries are added, and no entry is edited, removed, or reclassified once it exists. Nothing in the accompanying development transitions an assertion’s own classification from one state to another; each classification is computed once, from a fixed boundary specification and evidence context, and revising it means supplying a new context and computing again (Section 4’s monotonicity property covers exactly this case).

Remark 2.1 `Undefined` is a success state. A classifier that returns `Undefined` for an input outside its verified domain has computed the correct answer. The failure states of an audit classification system are not its `Undefined` outputs; they are its affirmative outputs on inputs it had no authority to classify.

3 Admissibility

Definition 3.1 (Admissibility) A material assertion a is admissible for an unqualified audit conclusion if and only if $\text{state}(a) = \text{Verified}$ under the declared boundary specification, and the classification was admitted by a verifier independent of the process that proposed it.

```
admissible(a) = true
  iff EvidenceState(a) = Verified
  and admitted_by(independent_verifier, a)
```

Both conjuncts are load-bearing. The first excludes every non-Verified state; the second excludes a Verified classification that a proposing process assigned to its own output, which is precisely the failure mode Theorem 8.3 rules out. In the accompanying development, `admitted.by(independent.verifier, a)` is not a bare predicate asserted true or false at the call site; it is the outcome of an explicit *admission certificate* check against a declared process registry. A certificate is accepted only if all of the following hold: it names the proposal it admits by that proposal’s identifier; the admitting process is present in the declared registry and holds verifier authority there; the proposing process is likewise present in the declared registry and holds proposer authority there; that proposing and admitting identities are distinct; the admission decision it carries is affirmative; and the proposal it admits itself claims Verified. Each of these six conditions has its own rejection case in the formal development (Section 8): a certificate naming the wrong proposal, one presented by an unregistered or non-verifying process, one naming an unregistered or non-proposing process, and one presented by the proposer itself are each independently ruled out. “Independent” in this paper means precisely this: a distinct, registered identity holding verifier authority in the declared registry, checked against the named proposal identifier. It does not mean, and this paper does not claim, that two distinct registered identities cannot be controlled by the same person or organisation, or that collusion between two honestly single-role identities is prevented; the registry establishes role and identity separation within itself, not organisational independence from anything outside it.

One further qualification belongs here rather than being left implicit. “Names the proposal it admits” and “the admitting process is present in the registry” are checks against identifiers – `ProposalId` and `ProcessId` – supplied by the caller, and this development does not itself establish that those identifiers are unique. If a caller allows two distinct proposals to carry the same `ProposalId`, a certificate drawn up for one validates against the other as well, since the check compares identifiers, not proposal contents; if a caller allows two distinct registry entries to share a `ProcessId`, registry lookup returns whichever entry happens to be found first, making its role- and identity-checks depend on list order rather than on a well-defined population. Both are exactly the kind of caller-supplied-input question this paper’s discipline otherwise declines to adjudicate (Section 9, Section 10): the boundary specification, the evidence context, and the materiality predicate are none of them checked for well-formedness either. The claim that a certificate names *the* proposal it admits, and that registry lookup identifies *the* process an identifier names, is accordingly conditional on the caller supplying unique `ProposalId` and `ProcessId` values; reusing one may permit certificate replay across proposals, or make registry interpretation depend on declaration order, and this development does not detect or reject either.

Presumptive states may guide triage. They order queues, allocate investigative attention, and inform planning. They may not support an unqualified conclusion. The moment a presumptive state binds consequence, the discipline has been abandoned, because a presumptive state is precisely a state whose evidence does not meet the declared grade.

Professional judgement is not denied by this rule; it is constrained in where it may act without leaving a trace. Judgement may establish or revise thresholds, materiality, evidence assessments, and boundary specifications – that is where it belongs, and nothing in this paper removes it from there. What it may not do

is depart, at the point of a single assertion's classification, from a condition the declared specification requires, without that departure itself being recorded as an explicit, versioned revision to the specification rather than a silent override applied after the evidence has disappointed. The boundary and admission machinery of this paper does not choose thresholds or assign materiality; it enforces that whatever thresholds and materiality judgements were declared are the ones actually applied.

4 Boundaries and Preconditions

Definition 4.1 (Boundary) A boundary is the declared region, in assertion class, jurisdiction, evidence grade, and materiality, inside which a verification procedure produces valid results. A verification procedure is valid only inside its declared boundary.

Every verification procedure carries a proof obligation of the same shape as a program statement: a precondition stating what must be true before the procedure is entitled to produce a verification result, and a postcondition stating exactly which classification the procedure is entitled to produce. The question to ask of every audit procedure is the weakest-precondition question: what must hold for this procedure's output to mean what it claims to mean?

```

Procedure: Direct bank confirmation

Preconditions:
  bank exists as independent counterparty
  confirmation route is controlled by the auditor
  jurisdiction permits direct confirmation
  account identifier is supplied
  response is received through an authenticated channel

Postcondition:
  the specified account-balance existence assertion, for the
  specified entity and date, may be classified Verified and
  may proceed to admission review

```

The postcondition is deliberately narrow in a second way, beyond stating a classification rather than a conclusion: direct bank confirmation, even fully satisfied, establishes only *existence* of the specified balance, for the specified entity and date. It does not establish rights and obligations (whether the entity actually controls the balance free of restriction), valuation (whether the recorded amount is correct at the relevant measurement basis), or completeness (whether all such balances have been identified). Each of those is a distinct assertion class with its own boundary specification and its own procedures; a single confirmation response does not discharge all of them merely because they concern the same underlying account, and a boundary specification that let it do so would itself be the kind of defect `EscalationRequired` (Section 2) exists to name.

If any precondition fails, the procedure cannot produce `Verified`. Not should not: cannot, within the declared system. A confirmation received through a route the auditor does not control, or from a jurisdiction that obstructs direct confirmation, does not weakly satisfy the precondition; it fails it, and the postcondition is unavailable. Silent conversion, in this vocabulary, is simply a postcondition violation:

a procedure whose preconditions failed was nonetheless treated as having discharged its postcondition.

Boundaries are declared as versioned, machine-readable data, so that the precondition check is itself mechanical. The declaration enumerates, for each procedure, the assertion classes it covers, the jurisdictions in which its preconditions are satisfiable, the evidence grade it produces, and the materiality that grade can support. An assertion for which no declared procedure’s preconditions hold is classified *Undefined* by construction.

5 Residuals and the Verification Balance Sheet

Anything outside the boundary is not erased. It is registered.

Definition 5.1 (Verification Balance Sheet) For a verification procedure V :

<code>VerificationBalanceSheet(V) =</code>	
<code>Assumptions(V)</code>	<code>what V takes as given without verification</code>
<code>Target(V)</code>	<code>exactly what successful completion establishes</code>
<code>Cost(V)</code>	<code>what completion consumes</code>
<code>Residuals(V)</code>	<code>what remains unestablished after completion</code>
<code>TrustChain(V)</code>	<code>the delegations connecting V to its assumptions</code>

subject to the constraint:

<code>Target(V)</code> must not include any claim in <code>Residuals(V)</code>
--

The constraint is the point. The verified target must not smuggle in its own residuals. A procedure whose stated target quietly includes claims that its own residual list disclaims has misstated what it verifies, and every downstream use of its output inherits the misstatement. Rule 1.1 is enforced here: the `Target` line is the exact proposition the procedure verifies, nothing more, and the `Residuals` line enumerates what a reader might wrongly assume is covered.

Definition 5.2 (Residual Register) Every classification other than an admitted *Verified* writes an entry to a residual register recording the assertion, the state assigned, the identity and version of the boundary specification that produced it, the identity of the evidence context it was evaluated against, a reason distinguishing why the entry was written (not classified *Verified*; classified *Verified* but missing a certificate; classified *Verified* but the certificate presented was invalid), and an identified owner for the residual risk or an explicit orphan flag triggering escalation. The register is append-only: no queue-clearing, phase-completion, or dashboard operation deletes or downgrades an entry.

Durability is what distinguishes a residual register from an exception log. An exception log records that something was flagged. A residual register records that something remains unestablished, and it continues to record this after the workflow that flagged it has closed.

This paper makes four separate claims about the register, not one, because they are different claims about different things and conflating them was an error in an earlier presentation of this discipline. *Emission*: every material dependency

that is not both classified **Verified** and validly admitted automatically writes a matching entry – this is a property of the classification pipeline (Section 8), not a convention a workflow implementer is trusted to follow. *Historical preservation*: an entry, once written, remains in the register’s full history under every subsequent workflow operation, proved in general for the append-only **step** relation. *Pipeline log extension*: one run of the pipeline over a list of dependencies only ever appends to whatever log it was given – a narrower, structural fact about the emission function itself, proved directly from its recursive definition rather than from the general **step** relation. There is no open/closed distinction between residual entries in this development for either claim to be about; both are about what the register’s history contains, not about which of its entries currently count as outstanding. *Orphan escalation*: an entry with no assigned owner is exactly what an escalation query over the register returns, not merely an entry that query happens to preserve once found. Section 8 states and proves each of these four separately.

One further claim needs to be made by its absence rather than its presence. This paper’s formal development does not model resolving or closing a residual: there is no state distinguishing an entry that has since been addressed from one that has not, and no certificate or event that would move an entry from open to closed. A workflow operation described informally as “clearing the dashboard” is representable in this development only as an operation that may append further entries; it is not representable as one that marks existing entries resolved, because no such marking exists to apply. Real audit workflows plainly do resolve residuals eventually, and a future extension of this discipline should model that explicitly, with its own certificate and its own soundness statement, rather than by weakening the append-only property this paper already has. Stating this gap here, rather than leaving a reader to infer it from what does not appear, is itself an instance of the discipline this paper argues for.

6 Composition

Audits are made of many procedures, and an opinion depends on many assertions. The discipline is worthless unless correct local classifications compose into a correct conclusion, so the composition rule is stated explicitly.

Rule 6.1 (Composition) *An audit opinion is admissible as unqualified only if every material assertion on which it depends is admissible: a material dependency that is not both verified and validly admitted blocks an unqualified decision, regardless of how many other dependencies are fine.*

```

if exists a in deps(Opinion) such that not admissible(a)
then Opinion is inadmissible as unqualified

(* admissible(a) is false whenever
   state(a) in {Undefined, Refuted, PresumptivelyVerified,
                PresumptivelyRefuted, EscalationRequired}
   or state(a) = Verified but admission was not independent *)

```

This paper and the accompanying development call this the *weakest-link* rule informally, because a Boolean conjunction fails as soon as any one conjunct fails,

in the same sense that a chain is no stronger than its weakest link. The name is colloquial rather than a formal claim: the composition rule is a Boolean conjunction over the **admissible** predicate (Section 3), not a statement that the six states of Section 2 form a total order in which one state can be compared as “weaker” than another in general. What the rule excludes is a specific, different failure: any construction that lets a preponderance of **Verified** classifications carry an **Undefined** or otherwise non-admissible one across the admissibility line, whether by averaging evidence quality across assertions or by any other aggregation that treats the conjunction as anything softer than exact. Such a construction would reintroduce the implicit default at the composition layer after eliminating it at the classification layer.

The kernel receives the set of dependencies already judged material, individually or in aggregate, as an input; it does not itself determine accounting materiality. That determination – which assertions are material to a given opinion, and at what threshold – is supplied by the caller, exactly as the boundary specification and the process registry are, and Section 9 and Section 10 return to what this paper does and does not claim about where that determination comes from.

7 Three Violations of the Invariant

The invariant was stated before any example, and the discipline stands whether or not the following cases had occurred. The cases are exhibited because each violates the invariant in a different layer: the first in engagement-level classification, the second in workflow, the third in application semantics.

7.1 Wirecard as a Classification Trace

The collapse of Wirecard AG in 2020 is commonly narrated as a failure of scepticism or an achievement of fraud (McCrum 2022; ESMA 2020). At the classification layer isolated in this paper, the failure can instead be represented as an unresolved cash-existence assertion being permitted to support an unqualified conclusion, and it can be exhibited as a trace. The fixture below is a formalised version of the published narrative, checked in the accompanying Coq/Rocq development (Section 8); it is not a reconstruction of Ernst & Young’s actual working papers, and every failed precondition it encodes is drawn from the same published sources cited above rather than from access to those papers.

Assertion: A = "Wirecard holds EUR 1.9bn in Philippine trustee accounts." Required boundary for material cash existence: direct bank confirmation, authenticated route, cooperative jurisdiction Observed evidence: no authenticated direct bank confirmation intermediary documentation (screenshots, trustee statements) jurisdictional obstruction Precondition check (direct bank confirmation):

```

confirmation route controlled by auditor .... FAILED
jurisdiction permits direct confirmation ... FAILED

Classification under the discipline:
state(A) = Undefined

Admissibility:
A inadmissible for an unqualified opinion    (Rule 1.2, Central Invariant)

Legacy outcome:
A treated as accepted; unqualified opinions issued

Violation:
Undefined(A) was converted into Accepted(A)

```

The claim made by this trace is deliberately narrow. The discipline would not have prevented the fraud, and it compels no particular exercise of judgement. It would have changed the classification state of assertion *A*: the evidential obstruction would have been recorded as *Undefined*, with a registered residual and an inadmissible unqualified opinion, rather than passing as ordinary acceptance. The jurisdictional, evidence-quality, and control-boundary failures that enabled the fraud are, in this vocabulary, three failed preconditions whose failure the legacy system did not record.

The accompanying development exercises five further variations on this same assertion, not only the one above, because *Undefined* is only one of several ways an unqualified opinion is ruled out. With every precondition satisfied, the assertion classifies *Verified* – the boundary specification is not vacuous – but that alone is not enough: presented with no admission certificate at all, the opinion remains inadmissible for want of one; presented with a certificate in which the proposing process names itself as its own admitting verifier, or names a proposer that is unregistered or registered with the wrong authority, the certificate is rejected – the first by exactly the mechanism of Theorem 8.3 – and the opinion remains inadmissible; only when a certificate from a distinct, correctly registered verifier admits a correctly registered proposer’s proposal does the opinion become admissible as unqualified. Section 8 states the outcome of all six fixtures precisely.

7.2 Continuous Auditing Queues as an Algorithm

The same violation occurs at ordinary scale in continuous auditing, and there it can be written as a three-line algorithm. Suppose, hypothetically, a monitoring system flags 10,000 transactions in a period and a thresholding layer routes 200 to human review; the accompanying formal fixture (Section 8) uses ten transactions and two reviewed, at a scale small enough for the proof assistant to evaluate directly, not to reproduce this ratio. The undisciplined system runs:

```

for each exception e:
  if priority(e) < threshold:
    clear(e)

```

The 9,800 cleared exceptions are not verified; they are deprioritised. If the dashboard then reports that 200 exceptions were reviewed with no material issue found, while the 9,800 vanish from consequence, the system has silently converted

unreviewed exceptions into accepted background. The disciplined system, matching the pipeline of Section 8 rather than introducing a state that pipeline does not have, runs:

```
for each exception e:
    classify(e)                -- Verified or Undefined, by construction
    if not (classify(e) = Verified and validly_admitted(e)):
        residual_register.add(e) -- automatic, not a separate step
                                a workflow author can omit
```

A deprioritised transaction is not assigned a separate *Unreviewed* state; it remains *Undefined*, exactly as classification against the declared boundary computes it (Section 2), and a residual entry recording that is emitted automatically rather than by a workflow author remembering to call `residual_register.add`. “Clearing the dashboard” is representable in this pipeline only as a further workflow operation that may append entries; it does not, and cannot, construct the resolution certificate that would be needed to mark an existing entry closed, because this development does not model one (Section 5). The governing fact is consequently not an inequality between two states but a preservation property: every residual entry the pipeline emits remains in the register after any such further operation. That single property is the continuous auditing case in full. Prioritisation is legitimate; capacity is finite; triage is not the error. The error is the disappearance of the deprioritised exceptions from the record of what remains unestablished. Under the discipline they persist in the residual register, and the aggregate of such entries is itself evidence about the assurance the monitoring system actually provides, as distinct from the assurance its cleared dashboard displays (Alles et al. 2006; Issa and Kogan 2014; Austin et al. 2021).

7.3 SQL Unknown as a Semantic Error

At the data layer, accounting information systems already possess the required third value, and three distinct things need to be kept apart here because the layer above the database routinely collapses them into one. *NULL* is a marker attached to a stored value, recording that no value is present; it is not itself a truth value. *Unknown* is the truth value SQL’s three-valued comparison semantics returns when a predicate is evaluated against a *NULL*, standardised behaviour rather than an implementation quirk (Codd 1979) and specified in the SQL standard itself (ISO/IEC 9075). Neither of these is the application-layer coercion that follows them: the numeric, Boolean, or string value some later layer substitutes for *Unknown* so that arithmetic, a conditional, or a report can proceed without a third case to handle. The database refuses to convert absence of information into an affirmative truth value; the coercion is introduced above it, by code the database’s own semantics did not require.

The error occurs above the database, and it is a semantic error with a precise description. The error is not that the database has an *Unknown* value. The error is that the surrounding system lacks the discipline to preserve it once the value leaves the database. A reporting layer that renders a null risk score as zero, a middleware coercion that aggregates a missing confirmation as no exception, and a filter predicate that silently excludes unresolved rows so that a completeness check passes are all instances of the same conversion:

Unknown at the data layer
must not become
Accepted at the audit layer

The accompanying development connects this case to the same boundary machinery every other fixture in this paper uses, rather than modelling it as an isolated two-value comparison: a nullable score becomes boundary evidence, present in the context if and only if the underlying value is present, so absence classifies Undefined through exactly the mechanism of Section 4, not through a special case written for this example. A general theorem in that development (Section 8) states that absence, represented this way, cannot reach an unqualified decision under *any* process registry, proposal, or admission certificate a caller might supply, not only the specific ones a demonstration happens to construct. The database example demonstrates that the technical capacity to represent verification failure has existed for decades. What has been missing is not the third value but the discipline that preserves it through every layer standing between the evidence and the conclusion.

8 Formal Development

The formalisation target is deliberately narrow and structural. The proof assistant is not verifying the audit. It is verifying an executable classification pipeline that prevents an unverifiable material assertion from being admitted as verified. Substantive testing, sampling sufficiency, valuation, and fraud detection are not formalised; the discipline constrains how their results are allowed to propagate into conclusions. This section states that pipeline precisely: its data structures, its five stages, the soundness theorem for each stage, and the end-to-end theorem composing them, in the same order the accompanying Coq/Rocq development states them.

Data structures

```
EqbSpec Fact      -- an equality test carrying its own correctness
                   proof (eqb_true_iff), not a bare function trusted
                   by convention
Procedure         -- id, name, preconditions, covers : Assertion -> bool
BoundarySpec     -- id, version, list of Procedure
ClassificationProposal -- id, assertion, claimed classification, proposer id
AdmissionCertificate -- proposal id it admits, verifier id, decision bit
DependencyPacket -- assertion, material?, classification, proposal,
                  certificate (option)
ResidualEntry    -- id, assertion, state, boundary id+version,
                  context id, reason, owner (option)
OpinionDecision  -- Unqualified
                  | InadmissibleOpinion (reasons : list
                    InadmissibilityReason)
PipelineInput    -- boundary spec, equality witness, context (+id),
                  assertions, materiality predicate, a proposal
                  identity and proposer per assertion, a
                  certificate function, process registry
AuditDecision    -- classifications, residuals, opinion decision
```

The pipeline

boundary classification	classify : BoundarySpec -> EqbSpec -> context -> Assertion -> EvidenceState
classification proposal	
admission validation	validate_admission : ProcessRegistry -> ClassificationProposal -> AdmissionCertificate -> bool
residual emission	process_dependency, classify_and_register
opinion decision	decide_opinion : ProcessRegistry -> list DependencyPacket -> OpinionDecision

`run_pipeline` composes all five stages over one `PipelineInput` into one `AuditDecision`. Each stage below is proved sound on its own, and the end-to-end theorem is a composition of those proofs, not an independent one.

Boundary classification

```

Definition classify {Fact Assertion : Type}
  (bspec : BoundarySpec Fact Assertion) (E : EqbSpec Fact)
  (ctx : list Fact) (a : Assertion) : EvidenceState :=
  if procedure_satisfied E ctx a (boundary_procedures bspec)
  then Verified else Undefined.

Theorem boundary_verified_has_witness :
  forall bspec E ctx a,
    classify bspec E ctx a = Verified ->
      exists p, In p (boundary_procedures bspec) /\
        procedure_covers p a = true /\
        has_all E ctx (procedure_preconditions p) = true.

```

`classify` is not primitive: its value is computed from the declared boundary specification and evidence context, exactly as Section 4 requires, and there is no third branch for it to take. `boundary_verified_has_witness` is the difference between a Boolean computation and a proof about what produced it: a `Verified` outcome comes with a concrete procedure, drawn from the declared specification, that covers the assertion and has every precondition present in the context – not merely that some computation happened to return `true`. A companion monotonicity theorem, `boundary_verified_context_monotone`, states that adding evidence never turns a `Verified` classification back to `Undefined`.

Admission validation

```

Definition validate_admission (registry : ProcessRegistry)
  (proposal : ClassificationProposal) (cert : AdmissionCertificate) : bool :=
  (cert names the correct proposal)
  && (the admitting process is registered with Verifier authority)
  && (the proposing process is registered with Proposer authority)
  && (the admitting process's identity differs from the proposer's)

```

```

&& (admission_decision cert)
&& (proposal_classification proposal = Verified).

```

(Set out here for readability; the accompanying development states the six conjuncts as a single Boolean expression over `ProcessRegistry`, `ClassificationProposal`, and `AdmissionCertificate`.) An earlier version of this development checked the admitting process's registration and role but not the proposing process's – `proposal_process` was accepted as whatever identifier a proposal happened to carry, unchecked against the registry; the third conjunct above closes that. Each of the six conjuncts now has its own rejection theorem: `certificate_for_wrong_proposal_rejected`, `non_verifier_certificate_rejected`, `non_proposer_certificate_rejected`, `proposer_certificate_rejected` (the mechanism inside the pipeline for what the declarative layer below states as Theorem 8.3), `decision_false_certificate_rejected`, and `unverified_proposal_certificate_rejected`. `valid_admission_implies_verified` and `valid_admission_implies_distinct_process` state what a passing certificate guarantees.

Checking the conjuncts is not sufficient by itself: a proposal is only ever trustworthy admission-wise if it is also the *correct* proposal for the assertion actually being classified. `build_packet` (the classification-proposal stage above) is what makes this hold: rather than accepting a complete `ClassificationProposal` from the caller, `PipelineInput` supplies only a proposal identity and a proposer per assertion, and `build_packet` constructs the proposal itself from the real assertion and the real, freshly computed classification. `build_packet_proposal_matches` states that the two agree structurally, for every `PipelineInput`: there is no longer a value through which a proposal for one assertion, or claiming one classification, could be attached to a dependency packet for a different assertion or a different classification.

Residual emission

```

Definition process_dependency (registry) (boundary id, version) (context id)
  (fresh id) (d : DependencyPacket)
  : EvidenceState * option ResidualEntry :=
  if dep_material d && not (dep_ok registry d)    (* material and not
                                                    Verified-and-admitted *)
  then (dep_state d, Some (a new ResidualEntry recording d's assertion,
                        state, boundary provenance, and reason))
  else (dep_state d, None).

Theorem nonadmitted_material_emits_residual :
  forall registry ids d,
    dep_material d = true -> dep_ok registry d = false ->
    exists e, (process_dependency ... d produces (Some e)) /\
      residual_assertion e = dep_assertion d /\
      residual_state e = dep_state d.

```

Emission is gated on `dep_material` explicitly, matching the “every *material* dependency” wording of Definition 5.2 exactly rather than a broader behaviour: an earlier version of this development emitted for every dependency failing `dep_ok` regardless of materiality, which was not what the definition, or `NON_CLAIMS.md`, actually claimed. `nonmaterial_emits_no_residual` states the excluded case directly.

Four separate results, matching Section 5's four separate claims: *Emission* (`nonadmitted_material_emits_residual` above, gated on materiality, and its converse

`admitted_verified_emits_no_open_residual`; `nonmaterial_emits_no_residual` states the excluded non-material case directly); *historical preservation* (`residual_preservation`, `residual_preservation_chain`: the register's `step` relation is purely additive); *pipeline log extension* (`classify_and_register_step`: one pipeline run's own emitted entries only ever extend the log it was given, proved directly rather than via `step`); *orphan escalation* (`orphan_escalation_sound`: an entry with no owner is exactly what the orphan query returns). No resolution or closure mechanism is formalised, as Section 5 already states; there is no fifth theorem here about closing a residual because there is no closure operation, and no open/closed distinction on an entry, to have a theorem about.

Opinion decision

```

Definition decide_opinion (registry) (deps : list DependencyPacket)
  : OpinionDecision :=
  let material_deps := filter dep_material deps in
  if forallb (dep_ok registry) material_deps
  then Unqualified
  else InadmissibleOpinion (a reason per failing dependency).

Theorem decide_opinion_unqualified_sound :
  forall registry deps,
    decide_opinion registry deps = Unqualified ->
    forall d, In d deps -> dep_material d = true ->
      dep_state d = Verified /\ dep_valid_admission registry d = true.

```

This – not the declarative predicate below – is the theorem that now justifies “by construction”: it is soundness of an executable function that a runtime classifier actually calls, not an unfolding of a proposition defined to already contain the desired property. Eight further theorems cover every way a dependency can block `Unqualified`: `undefined_blocks_unqualified`, `refuted_blocks_unqualified`, `presumptive_blocks_unqualified`, `escalation_blocks_unqualified`, `verified_without_admission_blocks_unqualified`, and `invalid_admission_blocks_unqualified`, together with the single generic lemma, `decide_opinion_blocked`, they are each corollaries of `all_valid_dependencies_allow_unqualified` is the converse, completeness direction: the check is not vacuously impossible to pass.

The end-to-end theorem

```

Theorem pipeline_unqualified_sound :
  forall inp start_id log,
    decision_opinion (run_pipeline inp start_id log) = Unqualified ->
    forall a, In a (pi_assertions inp) -> pi_material inp a = true ->
      classify (pi_bspec inp) (pi_eq inp) (pi_context inp) a = Verified /\
        dep_valid_admission (pi_registry inp) (build_packet inp a) = true.

```

Given the declared boundary specification, evidence context, assertion set, process registry, and admission certificates threaded through one `PipelineInput`, the pipeline cannot decide `Unqualified` unless every material assertion in scope

is classified `Verified` under the declared boundary *and* carries a validly admitted certificate. This is Theorem 8.1 made executable and is the paper’s central result; it is proved as a direct corollary of `decide_opinion_unqualified_sound` applied to the packets `run_pipeline` itself builds, which is the point of stating the earlier stages separately – composition of already-proved theorems, not a new proof asked to carry the whole claim.

The declarative layer

Alongside the executable pipeline above, the development retains a smaller, earlier declarative layer: `OpinionAdmissible`, a `Prop` defined as exactly the per-dependency admissibility check, and `default_free_admissibility`, which unfolds it.

```

Theorem default_free_admissibility :
  forall (O : Opinion),
    OpinionAdmissible O ->
      forall (a : Assertion),
        In a (deps O) -> material a ->
          state a = Verified /\ independently_admitted a.

Theorem residual_preservation :
  forall (log1 log2 : RegisterLog) (e : ResidualEntry),
    step log1 log2 -> In e log1 -> In e log2.

Theorem no_self_admission :
  forall (p : Process) (a : Assertion),
    proposes p a -> ~ admits p a.

```

Theorem 8.1 (Default-Free Admissibility) *For any audit opinion O , if O is unqualified, then every material assertion on which O depends is classified `Verified` under the declared boundary specification and admitted by an independent verifier.*

Corollary 8.1 (Contrapositive) *If any material assertion on which O depends is `Undefined`, then O is not admissible as an unqualified opinion.*

Theorem 8.2 (Residual Preservation) *Every non-affirmative classification of a material assertion writes an entry to the residual register, and no workflow operation deletes or downgrades an entry.*

Theorem 8.3 (No Self-Admission) *A proposed classification cannot admit itself. Admission is performed only by a verifier independent of the process that generated the proposal.*

These four statements are still proved and still true; they are no longer presented as the central result, because a `Prop` defined to already contain the desired property, however genuinely proved, is a different kind of claim from soundness of a decision procedure that computes. The predicate is named `independently_admitted` rather than `admitted` precisely because `Admitted` is a reserved proof-closing command in Coq/Rocq; naming a semantic predicate identically to that tactic is the kind of small inconsistency the discipline itself warns against, so the development avoids it throughout. `step` is the register’s transition relation under any workflow operation;

the theorem states that every such step is log-extending, never log-shrinking. `admits` is deliberately a relation on the process performing admission rather than on the assertion alone, so that `no_self_admission` cannot be discharged by a proposing process simply relabelling itself.

The force of the pipeline theorem above lies in what it excludes: given a fixed `PipelineInput`, there is no execution of `run_pipeline` that reaches `Unqualified` from a dependency set containing an `Undefined`, non-admitted, or invalidly-admitted material assertion. Whether that makes the type error of Section 1 “unrepresentable” depends on what is being asked to be unrepresentable. `OpinionDecision` is an ordinary data type: nothing stops a caller from constructing the value `Unqualified` directly, disconnected from any dependency check, any more than a type system stops a programmer from writing a wrong literal. What `pipeline_unqualified_sound` shows is narrower and is the claim this paper actually makes: no proof of admissibility is derivable, and no execution of `run_pipeline` produces `Unqualified`, under the declared specification without satisfying the check. That is a soundness property of a function, not an unconstructibility property of a type, and this paper states it as such throughout rather than as “unrepresentable.”

Proof dependencies, the trusted computing base, and extraction

The development’s modules depend on each other in one direction only: `Base` (the state vocabulary and stable identifiers) is required by `Boundary` (classification), which together with `Base` is required by `SelfAdmission` (admission certificates); `Admissibility` (the opinion decision) requires all three; `Residual` (emission) requires `Admissibility`; `Pipeline` (composition) requires all of the above. `Cases` exercises the pipeline on the three worked examples of Section 7, and `Extraction` compiles the `Type/bool`-valued decision procedures – `classify`, `validate_admission`, `decide_opinion`, `process_dependency/classify_and_register`, and `run_pipeline` – to OCaml. The `Prop`-valued specification predicates (`admissible`, `OpinionAdmissible`, `proposes`, `admits`, `step`) are not extracted: Coq erases `Prop` at extraction time because they carry no computational content, and that is correct here, not an omission – they are proved about the classifier, not run by it.

One further boundary is worth stating precisely because it is easy to overstate what `EqbSpec` (Section 4’s `classify` paragraph) buys at runtime. Inside Coq, an `EqbSpec` carries its equality test together with a proof that the test is sound and complete (`eqb_true_iff`); that proof is what rules out a defective test silently making every precondition appear satisfied, and it is checked once, statically, by the kernel. Extraction erases that proof – correctly, since it is a `Prop` – and what reaches the extracted OCaml classifier is the equality *function* alone, running as ordinary, unverified OCaml code like the rest of the extracted kernel. The correctness proof constrains which `EqbSpec` values are *constructible* in Coq; it says nothing further once the specific function inside one has been extracted and is executing.

The trusted computing base is, accordingly: the Coq/Rocq kernel’s type checker, which `coqchk` independently re-verifies against every compiled module; the extraction mechanism itself, which translates a checked `Type/bool`-valued term to OCaml without re-proving anything about the translation; the OCaml compiler; and the small hand-written CLI wiring that constructs `PipelineInput` values and prints whatever `run_pipeline` returns (Section 7’s fixtures, mirrored exactly in that CLI).

None of the last three are machine-checked, exactly as with any Coq development that extracts to a general-purpose language; the regression test described below checks observed agreement between the extracted executable and the proved Examples, which is testing, not proof, and does not close that gap.

What is proved, computed, and merely observed

Five different kinds of claim appear in this section and it matters which is which. *Definitional lemmas* (`classify_verified_iff`, `dep_ok_false_of_not_verified`) unfold a definition and establish nothing beyond it. *Soundness theorems* (`boundary_verified_has_witness`, `decide_opinion_unqualified_sound`, `pipeline_unqualified_sound`) say an executable function’s output implies a property about its inputs. *Preservation theorems* (`residual_preservation`, `boundary_verified_context_monotone`) say a property survives a transition. *Executable calculations* (the worked-case Examples of Section 7, each closed by `reflexivity` – Coq evaluating a concrete, closed term and checking the result matches, not a general argument) are checked once, for the specific inputs given, not generalised. *Regression tests* (`tools/fixture_check.py`, comparing the extracted CLI’s printed output against the same worked cases) check the compiled OCaml executable against the proved Coq Examples and can only ever find disagreement, never prove agreement holds beyond the cases it runs.

Implementation and artefact availability

The formal core (`Base.v` through `Pipeline.v`) is 1,329 non-blank lines, 39 Theorem/Corollary statements, and 10 further named Lemmas (49 named declarations in total); `Cases.v` adds the three worked examples of Section 7. Every one of the 49 is checked, individually, to depend on no axiom (`Print Assumptions`, `rocq/PrintAssumptions.v`: all 49 report “Closed under the global context”, not a curated subset of them), in addition to `coqchk`’s independent re-verification of every compiled module and a source-level check against `Admitted`, `admit`, and `Axiom`. `make verify` runs the full sequence – build, extraction, `coqchk`, the assumption check, the source check, and the regression test – and is what continuous integration runs on every push and pull request. In keeping with this journal’s emphasis on tool and proof replicability, the Coq/Rocq development and the OCaml classifier it extracts to accompany this submission at <https://github.com/dhwcmoore/partial-audit-classification>, as of commit `c4ad9ba` and its history; a tagged release is intended for the final-version stage. The evaluation results reported in Section 7 are illustrative traces rather than outputs of a run over a production dataset, and the repository’s `NON_CLAIMS.md` states this distinction explicitly so that reviewers do not mistake the Wirecard and continuous-auditing examples for empirical findings.

The worked cases

Wirecard (Section 7.1) is six fixtures over the same assertion and boundary specification, each a full `run_pipeline` call. Incomplete evidence classifies `Undefined`, a residual

is emitted, and the decision is `InadmissibleOpinion` with reason `NotVerified`. Complete evidence with no certificate presented classifies `Verified` but the decision is still `InadmissibleOpinion`, with reason `MissingCertificate` – distinguished, by the reason alone, from the first fixture, exercising the two-conjunct rule of Section 3 directly. Complete evidence with a certificate from a distinct, registered verifier decides `Unqualified`, with no residual left open. The remaining three fixtures each present a certificate that would otherwise validate, varying only the named proposer, and are each rejected with reason `InvalidCertificate`: naming the proposer as its own admitting verifier (`proposer_certificate_rejected` – Theorem 8.3 exhibited inside the pipeline, not only at the level of the `Role` type), naming a proposer identifier absent from the registry, and naming a registered proposer holding `Verifier` rather than `Proposer` authority (both `non_proposer_certificate_rejected`, the latter isolated from the former with a second registered verifier so the fixture tests one failure mode at a time). Continuous auditing (Section 7.2) runs all ten transactions through the same pipeline; two, reviewed and admitted by a distinct verifying process, decide `Verified` with `dep_ok`; the remaining eight are `Undefined` and each emits a residual automatically, and a further workflow `step` – “clearing the dashboard” – is proved, via `residual_preservation`, not to remove any of the eight. SQL Unknown (Section 7.3) routes score presence through `classify` like any other boundary evidence, so absence classifies `Undefined` by construction, and a general fixture shows absence cannot reach `Unqualified` under *any* registry, proposal identity, proposer, or certificate a caller might supply, applying `pipeline_failed_material_dependency_blocks_unqualified` rather than re-deriving the fact for this case specifically.

Note on a related submission

A companion manuscript, *Boundary Discipline for Accounting Information Systems: On the Correct Treatment of Undefined Audit Assertions*, has been submitted to the *International Journal of Digital Accounting Research*. Both manuscripts develop the same central invariant (Rule 1.2) and share the Coq/Rocq development described above; the proof artefact is shared, not duplicated – neither manuscript re-derives it independently. The division of technical content is: the present manuscript’s central contribution is the executable kernel itself – the formal semantics of Sections 2–6, the admission-certificate mechanism, the residual register’s transition system, the extraction to OCaml, and the soundness proofs and implementation evaluation of Section 8 – while the companion manuscript’s central contribution is the accounting information systems architecture built on top of the same kernel: organisational adoption, risk ownership and the Risk Assignment Register, standards mapping against the ISA suite, inspection measures, and the broader accounting and XBRL-reporting implications its Section 9 develops. Both manuscripts define a Verification Balance Sheet and a residual register (Section 5 here), because both need the concept; neither is exclusive to one manuscript, and this note does not claim otherwise. What is exclusive to each is the depth and audience of treatment just described. This relationship is disclosed so that editors and reviewers can evaluate the degree of overlap directly; in my own assessment the two manuscripts share an invariant and a proof artefact but not a contribution, and neither manuscript’s findings depend on prior acceptance of the other. Before submission, the corresponding author

will notify the editors of both journals of this companion submission, consistent with each journal’s policy on related or concurrent submissions; this note should be updated to record the date once that notification has been sent, rather than asserting it as already done.

9 What Existing Literatures Supply

The related literatures each recognise part of the problem, and this section’s claim is narrower than exhaustive novelty: these areas contain relevant mechanisms, individually well developed, but this paper has not found one that combines them into an audit-specific, executable classification and admission pipeline, and the literature search behind this section is not comprehensive enough to support the stronger claim that none exists. The typed state space of Section 2 is, in the vocabulary of this journal’s own abstract-interpretation literature, a coarse abstraction of a much richer evidential domain (Rustenholz et al. 2026), in the lineage of Cousot and Cousot’s lattice-theoretic semantics for approximate program properties (Cousot and Cousot 1977); the present paper does not develop that apparatus, and confines itself to the six-state abstraction of Section 2 because that is the granularity at which the admissibility question of Section 3 is decidable by inspection.

Audit theory distinguishes what an auditor establishes from what an auditor merely fails to refute, and treats evidential sufficiency as the determinant of supportable assurance (Mautz and Sharaf 1961; Knechel et al. 2013; Nelson 2009; Hurtt 2010). Its standards already state the overall objective this paper’s invariant serves at the engagement level – obtaining reasonable assurance before expressing an opinion (IAASB, ISA 200 2016) – and already provide the relevant vocabulary for insufficiency: ISA 500 defines audit evidence and its sufficiency and appropriateness, ISA 505 governs external confirmations (the standard direct bank confirmation in Section 4’s running example falls under), and ISA 530 governs audit sampling (IAASB, ISA 500 2016; IAASB, ISA 505 2016; IAASB, ISA 530 2016), with the modified opinion in ISA 705 as the output channel for a failure to obtain sufficient appropriate evidence (IAASB 2016). The empirical literature shows that channel operating under judgement and incentive rather than under any classification architecture (Habib 2013). The recognition is real and, as Section 1 now states explicitly, this paper does not claim otherwise; the gap is that nothing in the classification *logic* surrounding these standards forces the non-affirmative channel to be taken rather than silently bypassed at the point a workflow needs a binary output.

Accounting information systems research supplies continuous monitoring, exception processing, and analytics (Vasarhelyi and Halper 1991; Kogan et al. 1999; Alles et al. 2006; Austin et al. 2021), and it documents the workflow conditions under which deprioritised exceptions are operationally absorbed (Issa and Kogan 2014). It lacks a state in which a deprioritised exception durably remains unresolved after the dashboard has been cleared.

Formal semantics supplies the underlying machinery for partiality itself. Kleene’s strong three-valued logic gives a truth-functional account of an undetermined value propagating through composition (Kleene 1952), and Belnap’s four-valued logic separates *unknown* from *contradictory* as distinct non-classical values rather than

collapsing both to one (Belnap 1977); the six-state vocabulary of Section 2 is coarser than either and is not offered as a refinement of them, but the propagation discipline of Section 6 is the same kind of claim they make formal. Scott’s domain-theoretic treatment of partial functions via a lifted, ordered value space is the classical semantic account of exactly the totalisation Section 1 describes informally (Scott 1970); option and result types in modern programming languages are its engineering descendants, and this paper’s `Undefined` plays the same structural role their `None/Nothing` constructor does, made audit-specific. Hoare’s axiomatic treatment of program correctness supplies the precondition/postcondition vocabulary Section 4 borrows directly (Hoare 1969), and design-by-contract practice built on it is the closest existing discipline to this paper’s verification-procedure obligations – applied there to program correctness, not to audit evidence. Proof-carrying code establishes that a producer can attach a checkable certificate to a claim so that a consumer need not trust the producer, only verify the certificate (Necula 1997); this paper’s admission certificates (Section 3) are a narrower, audit-specific instance of the same idea, checked against a declared registry rather than against a general-purpose verification condition. Separation-of-duty models in computer security formalise the requirement that no single actor complete a sensitive transaction alone (Clark and Wilson 1987); `validate_admission`’s distinct-identity requirement (Section 8) is the same requirement, specialised to classification and admission rather than to a well-formed transaction in general, and inherits the same limitation: it separates roles within a declared system, not organisational independence outside it.

Database theory supplies the third value itself (Codd 1979) and preserves it at the data layer, but permits every layer above to destroy it. Runtime verification supplies multi-valued verdicts over incomplete observations – a monitor rendering a verdict on a finite trace before the full, possibly infinite, behaviour is known – with inconclusive as a first-class outcome and presumptive refinements that guide without binding (Bauer et al. 2011, 2010; Leucker and Schallhart 2009), but this machinery has not been applied to audit evidence. Recent work on quantitative robustness for temporal specifications continues this line by replacing a binary verdict with a graded distance to the satisfaction boundary (Rino et al. 2026); that measure is a refinement of the *qualitative* verdict, useful for triage in exactly the sense Section 3 grants to presumptive states, and it is not itself an admissibility criterion, which is the distinction this paper insists on.

Each literature holds a piece, and none of the pieces above is this paper’s contribution by itself. What this paper adds is the combination: a typed classification of audit assertions in which `Undefined` is preserved from the point of verification failure through composition to audit consequence by an executable, machine-checked pipeline, with certificate-based admission and automatic residual emission as load-bearing parts of that pipeline rather than separate concerns.

10 Implications

For accounting information systems, the discipline is an architecture: claim packets carrying assertions and evidence, boundary specifications as versioned data, a classification function returning the typed states of Section 2, admission certificates checked against a declared process registry rather than an admission verifier merely separated by convention, and an append-only residual register. The architecture is

fail-closed: an assertion for which no declared procedure’s preconditions hold cannot reach an affirmative state by any path. Nothing in this architecture authenticates the evidence a procedure receives, determines the materiality threshold at which an assertion becomes subject to it, or removes the professional judgement that sets both; it constrains how the judgements that are declared are permitted to propagate into a conclusion, and no further.

For standards and inspection, the discipline yields measurable indicators rather than declared policy: the fraction of material assertions falling within declared boundaries, the materiality resting in regions whose correct classification is `Undefined`, and the fraction of residuals with an identified owner. A standard could require the declarations and the register; an inspection could measure the indicators; illustrative requirement language is confined to Appendix A, offered as illustrative consequences of the model rather than proposed regulatory text, because standards are administration and the contribution of this paper is the discipline. Nothing in this paper constitutes such a standard, establishes legal sufficiency of an opinion produced under it, or reflects adoption by any auditing body, standard-setter, or regulator; the appendix states requirements a standard *could* impose, not one that does. Once residuals are explicit they can also be priced, insured, or disclosed, but pricing is a consequence of the discipline, not a component of it.

For AI-assisted audit tools, the discipline becomes more urgent without becoming different. A generated conclusion is a proposed classification. Under Theorem 8.3 it cannot admit itself, and under Rule 1.2 it cannot bind consequence until an independent verifier, registered and distinct from the generating process, admits it `Verified` within a declared boundary via the certificate mechanism of Section 8. This is not a hypothetical caution: a recent benchmark of large language models on semantic-equivalence judgments over short code samples found substantial misclassification rates even with supporting context (Laneve et al. 2026), which is exactly the failure mode Theorem 8.3 is positioned against if a model’s own output were permitted to double as its admission. The treatment of generation as proposal requiring admission is the entire extension, and its development belongs to separate work.

11 Conclusion

No method prevents fraud. The object of this method is not to make fraud impossible. It is narrower, and it is what this paper has actually shown: given the declared boundary specification, evidence context, material dependency set, process registry, and admission certificates supplied to it, the extracted decision procedure of Section 8 cannot return `Unqualified` unless every material dependency is classified `Verified` *and* carries a valid admission certificate; every other material result generates a durable residual entry that no subsequent workflow operation removes.

That claim is exactly as narrow as its premises. This paper does not establish that the evidence supplied to a boundary procedure is genuine, that a materiality threshold declared to the kernel is the correct one, that two distinct registered process identities are free of collusion or organisational dependence, or that the extraction from a checked Coq proof to a running OCaml executable is itself machine-checked rather than trusted in the ordinary way any compiled artefact

is trusted. What has been verified is exactly what each procedure's postcondition states, nothing more; what has not been verified is registered rather than erased; and what admits a conclusion is a certificate checked against a declared registry, not a role asserted at the point of convenience. An audit system built to this discipline is humbler than its predecessors in precisely one respect: it knows when it is not entitled to speak, and it says so in a form a machine can check.

References

- Alles, M. G., Brennan, G., Kogan, A., and Vasarhelyi, M. A. (2006). Continuous monitoring of business process controls: A pilot implementation of a continuous auditing system at Siemens. *International Journal of Accounting Information Systems*, 7(2), 137–161. <https://doi.org/10.1016/j.accinf.2005.10.004>
- Austin, A. A., Carpenter, T. D., Christ, M. H., and Nielson, C. S. (2021). The data analytics journey: Interactions among auditors, managers, regulation, and technology. *Contemporary Accounting Research*, 38(3), 1888–1924. <https://doi.org/10.1111/1911-3846.12680>
- Bauer, A., Leucker, M., and Schallhart, C. (2010). Comparing LTL semantics for runtime verification. *Journal of Logic and Computation*, 20(3), 651–674. <https://doi.org/10.1093/logcom/exn075>
- Bauer, A., Leucker, M., and Schallhart, C. (2011). Runtime verification for LTL and TLTL. *ACM Transactions on Software Engineering and Methodology*, 20(4), Article 14. <https://doi.org/10.1145/2000799.2000800>
- Belnap, N. D. (1977). A useful four-valued logic. In J. M. Dunn and G. Epstein (Eds.), *Modern Uses of Multiple-Valued Logic*, 5–37. Dordrecht: Reidel. https://doi.org/10.1007/978-94-010-1161-7_2
- Clark, D. D. and Wilson, D. R. (1987). A comparison of commercial and military computer security policies. In *Proceedings of the 1987 IEEE Symposium on Security and Privacy*, 184–194. <https://doi.org/10.1109/SP.1987.10001>
- Codd, E. F. (1979). Extending the database relational model to capture more meaning. *ACM Transactions on Database Systems*, 4(4), 397–434. <https://doi.org/10.1145/320107.320109>
- Cousot, P. and Cousot, R. (1977). Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints. In *Proceedings of the 4th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages (POPL)*, 238–252. <https://doi.org/10.1145/512950.512973>
- European Securities and Markets Authority (2020). *Fast Track Peer Review on the Application of the Guidelines on the Enforcement of Financial Information by BaFin and FREP in the Context of Wirecard*. ESMA42-111-5349. Paris: ESMA. No DOI assigned.
- Habib, A. (2013). A meta-analysis of the determinants of modified audit opinion decisions. *Managerial Auditing Journal*, 28(3), 184–216. <https://doi.org/10.1108/02686901311304349>
- Hoare, C. A. R. (1969). An axiomatic basis for computer programming. *Communications of the ACM*, 12(10), 576–580. <https://doi.org/10.1145/363235.363259>
- Hurttt, R. K. (2010). Development of a scale to measure professional skepticism. *Auditing: A Journal of Practice & Theory*, 29(1), 149–171. <https://doi.org/10.2308/aud.2010.29.1.149>
- International Auditing and Assurance Standards Board (2016). *International Standard on Auditing 200 (Revised and Redrafted): Overall Objectives of the Independent Auditor and the Conduct of an Audit in Accordance with International Standards on Auditing*. In *2016–2017 Handbook of International Quality Control, Auditing, Review, Other Assurance, and Related Services Pronouncements*. New York: IFAC. ISBN 978-1-60815-318-3. No DOI assigned.
- International Auditing and Assurance Standards Board (2016). *International Standard on Auditing 500 (Redrafted): Audit Evidence*. In *2016–2017 Handbook of International Quality Control, Auditing, Review, Other Assurance, and Related Services Pronouncements*. New York: IFAC. ISBN 978-1-60815-318-3. No DOI assigned.
- International Auditing and Assurance Standards Board (2016). *International Standard on Auditing 505 (Revised and Redrafted): External Confirmations*. In *2016–2017 Handbook of*

- International Quality Control, Auditing, Review, Other Assurance, and Related Services Pronouncements*. New York: IFAC. ISBN 978-1-60815-318-3. No DOI assigned.
- International Auditing and Assurance Standards Board (2016). *International Standard on Auditing 530 (Redrafted): Audit Sampling*. In *2016–2017 Handbook of International Quality Control, Auditing, Review, Other Assurance, and Related Services Pronouncements*. New York: IFAC. ISBN 978-1-60815-318-3. No DOI assigned.
- International Auditing and Assurance Standards Board (2016). *International Standard on Auditing 705 (Revised): Modifications to the Opinion in the Independent Auditor's Report*. New York: IFAC. No DOI assigned.
- Issa, H. and Kogan, A. (2014). A predictive ordered logistic regression model as a tool for quality review of control risk assessments. *Journal of Information Systems*, 28(2), 209–229. <https://doi.org/10.2308/isys-50808>
- Kleene, S. C. (1952). *Introduction to Metamathematics*. Amsterdam: North-Holland. No DOI assigned.
- Knechel, W. R., Krishnan, G. V., Pevzner, M., Shefchik, L. B., and Velury, U. K. (2013). Audit quality: Insights from the academic literature. *Auditing: A Journal of Practice & Theory*, 32(Supplement 1), 385–421. <https://doi.org/10.2308/ajpt-50350>
- Kogan, A., Sudit, E. F., and Vasarhelyi, M. A. (1999). Continuous online auditing: A program of research. *Journal of Information Systems*, 13(2), 87–103. <https://doi.org/10.2308/jis.1999.13.2.87>
- Laneve, C., Spanò, A., Ressi, D., et al. (2026). Understanding code semantics: a benchmark study of LLMs. *International Journal on Software Tools for Technology Transfer*, 28, 329–343. <https://doi.org/10.1007/s10009-026-00842-4>
- Leucker, M. and Schallhart, C. (2009). A brief account of runtime verification. *The Journal of Logic and Algebraic Programming*, 78(5), 293–303. <https://doi.org/10.1016/j.jlap.2008.08.004>
- Mautz, R. K. and Sharaf, H. A. (1961). *The Philosophy of Auditing*. Sarasota, FL: American Accounting Association.
- McCrum, D. (2022). *Money Men: A Hot Startup, a Billion Dollar Fraud, a Fight for the Truth*. London: Bantam Press.
- Necula, G. C. (1997). Proof-carrying code. In *Proceedings of the 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, 106–119. <https://doi.org/10.1145/263699.263712>
- Nelson, M. W. (2009). A model and literature review of professional skepticism in auditing. *Auditing: A Journal of Practice & Theory*, 28(2), 1–34. <https://doi.org/10.2308/aud.2009.28.2.1>
- Rino, N., Foughali, M., Renkin, F., and Asarin, E. (2026). Efficiently computable temporal robustness for a practical STL fragment. *International Journal on Software Tools for Technology Transfer*, 28, 71–103. <https://doi.org/10.1007/s10009-026-00852-2>
- Rustenholtz, L., Lopez-Garcia, P., and Hermenegildo, M. V. (2026). Abstractions of sequences, functions and operators. *International Journal on Software Tools for Technology Transfer*, 28, 277–301. <https://doi.org/10.1007/s10009-026-00843-3>
- Scott, D. (1970). Outline of a mathematical theory of computation. Technical Monograph PRG-2, Oxford University Computing Laboratory, Programming Research Group. No DOI assigned.
- Vasarhelyi, M. A. and Halper, F. B. (1991). The continuous audit of online systems. *Auditing: A Journal of Practice & Theory*, 10(1), 110–125. No DOI assigned.

A Illustrative Requirement Language

The following requirements illustrate how the discipline could be codified in a standard, an inspection protocol, or an internal methodology. They are illustrative rather than proposed regulatory text.

1. Verification boundaries must be declared for all material assertion classes as versioned, machine-readable data specifying procedures, jurisdictions, evidence grades, and supportable materiality, together with the precondition set of each procedure.
2. Classification of assertions against declared boundaries must be mechanical, and assertions for which no declared procedure's preconditions hold must be classified **Undefined**.

3. Every non-affirmative classification of a material assertion must write a durable entry to a residual register identifying the residual and its owner, with orphaned residuals escalated.
4. An unqualified conclusion is admissible only if every material assertion on which it depends is an admitted **Verified** classification, where admission is performed by a verifier independent of every proposing process.
5. Inspection must measure boundary coverage, undefined-region materiality, and residual assignment from the declared specifications and classification logs rather than from declared policy.